Practical week-9

Task-1 : Implement a file copy utility using low-level Linux file I/O system calls (open, read, write, lseek, close). Compare its performance with standard library functions (fopen, fread, fwrite).

```
samanth-ubuntu@samanth-ubuntu:~$ mkdir file_copy_practical
samanth-ubuntu@samanth-ubuntu:~$ cd file_copy_practical
samanth-ubuntu@samanth-ubuntu:~/file_copy_practical$ pwd
/home/samanth-ubuntu/file_copy_practical
samanth-ubuntu@samanth-ubuntu:~/file_copy_practical$ nano source.txt
```

```
  GNU nano 7.2                                                  source.txt
This is a sample file for testing file copy operations.
Linux provides low-level system calls for file handling.
We will compare low-level I/O with standard library I/O.
```

PART A:

```
samanth-ubuntu@samanth-ubuntu:~/file_copy_practical$ nano lowlevel_copy.c
samanth-ubuntu@samanth-ubuntu:~/file_copy_practical$ gcc lowlevel_copy.c
-o lowlevel_copy
samanth-ubuntu@samanth-ubuntu:~/file_copy_practical$ ./lowlevel_copy
Low-level file copy completed.
Time taken: 12.00 microseconds
samanth-ubuntu@samanth-ubuntu:~/file_copy_practical$ ls
lowlevel_copy  lowlevel_copy.c  lowlevel_copy.txt  source.txt
samanth-ubuntu@samanth-ubuntu:~/file_copy_practical$ cat lowlevel_copy.txt
This is a sample file for testing file copy operations.
Linux provides low-level system calls for file handling.
We will compare low-level I/O with standard library I/O.
samanth-ubuntu@samanth-ubuntu:~/file_copy_practical$
```

```
  GNU nano 7.2
#include <stdio.h>
#include <fcntl.h>
#include <unistd.h>
#include <stdlib.h>
#include <sys/time.h>

int main()
{
    int src, dest;
    char buffer[1024];
    ssize_t bytesRead, bytesWritten;
    struct timeval start, end;

    src = open("source.txt", O_RDONLY);

    if (src < 0)
    {
```

```
GNU nano 7.2
        perror("Error opening source file");
        return 1;
    }

    dest = open("lowlevel_copy.txt", O_WRONLY | O_CREAT | O_TRUNC, 0644);

    if (dest < 0)
    {
        perror("Error opening destination file");
        close(src);
        return 1;
    }

    gettimeofday(&start, NULL);

    while ((bytesRead = read(src, buffer, sizeof(buffer))) > 0)
    {
        bytesWritten = write(dest, buffer, bytesRead);

        if (bytesWritten != bytesRead)
        {
            perror("Error writing file");
            close(src);
            close(dest);
            return 1;
        }
    }

    lseek(src, 0, SEEK_SET);

    gettimeofday(&end, NULL);

    close(src);
    close(dest);

    double time_taken =
        (end.tv_sec - start.tv_sec) * 1000000.0 +
        (end.tv_usec - start.tv_usec);

    printf("Low-level file copy completed.\n");
    printf("Time taken: %.2f microseconds\n", time_taken);

    return 0;
}
```

PART B:

```
samanth-ubuntu@samanth-ubuntu:~/file_copy_practical$ nano standard_copy.c
samanth-ubuntu@samanth-ubuntu:~/file_copy_practical$ gcc standard_copy.c -o standard_copy
samanth-ubuntu@samanth-ubuntu:~/file_copy_practical$ ./standard_copy
Standard library file copy completed.
Time taken: 36.00 microseconds
samanth-ubuntu@samanth-ubuntu:~/file_copy_practical$ ls -l
total 52
-rwxr-xr-x 1 samanth-ubuntu samanth-ubuntu  16360 Sep 26 08:53 lowlevel_copy
-rw-r--r-- 1 samanth-ubuntu samanth-ubuntu   1224 Sep 26 08:53 lowlevel_copy.c
-rw-r--r-- 1 samanth-ubuntu samanth-ubuntu    170 Sep 26 08:53 lowlevel_copy.txt
-rw-r--r-- 1 samanth-ubuntu samanth-ubuntu    170 Sep 26 08:52 source.txt
-rwxr-xr-x 1 samanth-ubuntu samanth-ubuntu  16368 Sep 26 08:56 standard_copy
-rw-r--r-- 1 samanth-ubuntu samanth-ubuntu    979 Sep 26 08:56 standard_copy.c
-rw-r--r-- 1 samanth-ubuntu samanth-ubuntu    170 Sep 26 08:56 standard_copy.txt
samanth-ubuntu@samanth-ubuntu:~/file_copy_practical$ cmp source.txt lowlevel_copy.txt
samanth-ubuntu@samanth-ubuntu:~/file_copy_practical$ cmp source.txt standard_copy.txt
samanth-ubuntu@samanth-ubuntu:~/file_copy_practical$ diff source.txt lowlevel_copy.txt
samanth-ubuntu@samanth-ubuntu:~/file_copy_practical$ diff source.txt standard_copy.txt
samanth-ubuntu@samanth-ubuntu:~/file_copy_practical$ ./lowlevel_copy
Low-level file copy completed.
Time taken: 10.00 microseconds
Standard library file copy completed.
Time taken: 7.00 microseconds
```

GNU nano 7.2 standard_copy.c

```c
#include <stdio.h>
#include <stdlib.h>
#include <sys/time.h>

int main()
{
    FILE *src, *dest;
    char buffer[1024];
    size_t bytesRead;
    struct timeval start, end;

    src = fopen("source.txt", "rb");

    if (src == NULL)
    {
        perror("Error opening source file");
        return 1;
    }

    dest = fopen("standard_copy.txt", "wb");

    if (dest == NULL)
    {
        perror("Error opening destination file");
        fclose(src);
        return 1;
    }

    gettimeofday(&start, NULL);

    while ((bytesRead = fread(buffer, 1, sizeof(buffer), src)) > 0)
    {
        fwrite(buffer, 1, bytesRead, dest);
    }

    fseek(src, 0, SEEK_SET);

    gettimeofday(&end, NULL);

    fclose(src);
    fclose(dest);

    double time_taken =
        (end.tv_sec - start.tv_sec) * 1000000.0 +
        (end.tv_usec - start.tv_usec);

    printf("Standard library file copy completed.\n");
    printf("Time taken: %.2f microseconds\n", time_taken);

    return 0;
}
```

Task 2 : Develop a program that redirects standard input and output using file descriptors and dup2(). Demonstrate how shells implement I/O redirection.

```
samanth-ubuntu@samanth-ubuntu:~$ mkdir redirect_practical
samanth-ubuntu@samanth-ubuntu:~$ cd redirect_practical
samanth-ubuntu@samanth-ubuntu:~/redirect_practical$ nano input.txt
samanth-ubuntu@samanth-ubuntu:~/redirect_practical$ cat input.txt
Hello Linux!
This is input redirection.
dup2() is used for file descriptor redirection.
```

```
samanth-ubuntu@samanth-ubuntu:~/redirect_practical$ nano redirect.c
samanth-ubuntu@samanth-ubuntu:~/redirect_practical$ ls
input.txt  redirect.c
samanth-ubuntu@samanth-ubuntu:~/redirect_practical$ gcc redirect.c -o redirect
samanth-ubuntu@samanth-ubuntu:~/redirect_practical$ ./redirect
Input and output redirection completed.
Input file: input.txt
Output file: output.txt
samanth-ubuntu@samanth-ubuntu:~/redirect_practical$ ls
input.txt  output.txt  redirect  redirect.c
samanth-ubuntu@samanth-ubuntu:~/redirect_practical$ cat output.txt
Hello Linux!
This is input redirection.
dup2() is used for file descriptor redirection.
```

```
 GNU nano 7.2
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <fcntl.h>
#include <sys/wait.h>

int main()
{
    int input_fd, output_fd;
    pid_t pid;

    input_fd = open("input.txt", O_RDONLY);

    if (input_fd < 0)
    {
        perror("Error opening input file");
        return 1;
    }

    output_fd = open("output.txt", O_WRONLY | O_CREAT | O_TRUNC, 0644);

    if (output_fd < 0)
    {
```

```c
    {
        perror("Error opening output file");
        close(input_fd);
        return 1;
    }

    pid = fork();

    if (pid < 0)
    {
        perror("Fork failed");
        return 1;
    }

    if (pid == 0)
    {
        dup2(input_fd, STDIN_FILENO);

        dup2(output_fd, STDOUT_FILENO);

        close(input_fd);
        close(output_fd);

        execlp("cat", "cat", NULL);

        perror("execlp failed");
        exit(1);
    }
    else
    {
        close(input_fd);
        close(output_fd);

        wait(NULL);

        printf("Input and output redirection completed.\n");
        printf("Input file: input.txt\n");
        printf("Output file: output.txt\n");
    }

    return 0;
}
```